

Files and File Handling in Python

What is a File?

A **file** is a named location on a storage device used to permanently store data.

Examples include:

- `students.txt` — stores text data.
- `vlog.mp4` — stores video.
- `photo.jpg` — stores an image.
- `program.py` — stores Python code.

Unlike variables, which generally hold data temporarily in memory, files allow data to be **stored permanently** and accessed later.

What is File Handling?

File handling is the process of creating, opening, reading, writing, updating, and closing files using a programming language.

In Python, file handling is mainly performed using the built-in `open()` function.

The `open()` function in Python is a [built-in function](#) used to **connect your script to a file** on your system, allowing you to read, write, modify, or create files.

Syntax:

```
file = open("filename.txt", "mode")
```

A **file mode** tells Python what you want to do with a file when you open it.

Mode	Meaning	If file does not exist	Existing data
r	Read	Error	Preserved
w	Write	Creates file	Deleted/overwritten
a	Append	Creates file	Preserved
x	Create	Creates file	Error if already exists
r+	Read + Write	Error	Preserved
w+	Write + Read	Creates file	Deleted/overwritten
a+	Append + Read	Creates file	Preserved

1. **r** — Read Mode

Used when you only want to **read** an existing file.

```
with open("data.txt", "r") as file:  
    data = file.read()  
    print(data)
```

If `data.txt` does not exist, Python raises [FileNotFoundError](#).

2. **w** — Write Mode

Used to **write new data**.

```
with open("data.txt", "w") as file:  
    file.write("Hello Python")
```

If the file already contains data, **w removes the old content** and writes the new content.

For understanding...

For example, if the file contains:

Hello
World

and you execute:

```
with open("data.txt", "w") as file:  
    file.write("Python")
```

The file becomes:

Python

Important ⚠️⚠️

If the file **does not exist** and you open it in **write (w) mode**, Python will **automatically create the file**.

3. **a** — Append Mode

Used to **add data at the end** of a file.

```
with open("data.txt", "a") as file:  
    file.write("\nPython")
```

If the file already contains:

Hello
World

it becomes:

Hello
World
Python

The old content is preserved.

4. **x** — Create Mode

Used specifically to **create a new file**.

```
with open("newfile.txt", "x") as file:  
    file.write("New file created")
```

IMPORTANT:

If `newfile.txt` already exists, Python raises:

FileExistsError

Mode	File exists	File does not exist
<code>r</code>	Opens it	 Error
<code>w</code>	Opens and overwrites it	 Creates it
<code>a</code>	Opens and adds at end	 Creates it
<code>x</code>	 Error	 Creates it

Python provides several **built-in file methods** to interact with the file's content and structure.

Method	Syntax Example	Return Value	Primary Use Case
<code>read()</code>	<code>file.read(5)</code>	<code>str</code> or <code>bytes</code>	Reads the entire file, or a specific number of characters/bytes.
<code>readline()</code>	<code>file.readline()</code>	<code>str</code> or <code>bytes</code>	Reads the next single line, including the trailing <code>\n</code> .
<code>readlines()</code>	<code>file.readlines()</code>	<code>list</code> of <code>strings</code>	Reads all remaining lines and packs them into a Python list.
<code>write()</code>	<code>file.write("text")</code>	<code>int</code> (char count)	Writes a single string directly into the file.
<code>writelines()</code>	<code>file.writelines(list)</code>	<code>None</code>	Writes a list of strings sequentially without adding newlines.
<code>tell()</code>	<code>file.tell()</code>	<code>int</code> (byte index)	Locates the exact current position of the cursor.
<code>seek()</code>	<code>file.seek(0)</code>	<code>int</code> (new index)	Moves the cursor to a specific byte location in the file.

File Pointer

A **file pointer** is a position indicator that keeps track of **where Python is currently reading or writing inside a file**.

You can think of it like a **cursor** in a text editor. The cursor tells you where the next character will be entered; similarly, the file pointer tells Python where the next read or write operation will take place.

tell() → tells the current position of the file pointer.

seek() → moves the file pointer to a specified position.

Suppose the file contains:

```
Hello
```

The positions are:

Character:	H	e	l	l	o
Position:	0	1	2	3	4

But the **file pointer can also be at position 5**, which is just after the last character:

Character:	H	e	l	l	o	
Position:	0	1	2	3	4	5

↑
End of file

```
# Open the file in read mode.
with open("data.txt", "r") as file:

    # Show the current pointer position.
    print(file.tell())

    # Read 5 characters.
    print(file.read(5))

    # Show the new pointer position.
    print(file.tell())

    # Move the pointer back to the beginning.
    file.seek(0)

    # Read again from the beginning.
    print(file.read(5))
```

Output

```
0
Hello
5
Hello
```

`tell()` → tells the pointer position

`seek()` → moves the pointer

`read()` → reads from the current pointer position

2. Line-by-Line Reading

Line-by-line reading means reading a file **one line at a time** instead of **reading the entire file at once**.

It is useful when working with large files because only one line needs to be processed at a time.

For example, suppose `students.txt` contains:

Rahul
Amit
Priya
Neha
Karan

We can read it line by line using a `for` loop:

```
with open("students.txt", "r") as file:
```

```
    # Read one line at a time.
```

```
    for line in file:
```

```
        # Remove the newline character.
```

```
        print(line.strip())
```

Python automatically moves the **file pointer** to the next line after reading each line.

Examples

1) Write a Python program to write and read the names of students from a file.

Create a file named `students.txt` and write the following names into it:

Rahul Amit Priya Neha Karan

Then read the contents of the file and display all the names on the screen.

2) Write a Python program to read `students.txt` **line by line** and display each student's name separately. (concept of loop will be used)

3) Write a Python program to add a new student named `Arjun` at the end of `students.txt` without deleting the existing names.

4) Write a Python program to count and display the total number of lines in `students.txt`. (loop + counter variable example)

5) file pointer concept

Create a file named `data.txt` containing `Hello Python`.

Write a Python program to display the initial file pointer position, read the first 5 characters, display the new pointer position, move the pointer back to the beginning using `seek()`, and read the first 5 characters again.

Write a Python program to read a text file and count the total number of words present in it.

```
# Open the file in read mode.
with open("data.txt", "r") as file:

    # Read the complete file.
    data = file.read()

# Split the text into individual words.
words = data.split()

# Display the number of words.
print("Total words:", len(words))
```

`readline()` reads one line at a time from a file.

`</>` Python

```
# Open the file in read mode.  
with open("students.txt", "r") as file:  
  
    # Read the first line.  
    line = file.readline()  
  
    # Display the line.  
    print(line)
```

If the file contains:

```
Rahul  
Amit  
Priya
```

Output:

```
Rahul
```

Write a Python program to read all the lines from `students.txt` using `readlines()` and display them one by one.

```
# Open the file in read mode.  
with open("students.txt", "r") as file:  
  
    # Read all lines and store them in a list.  
    lines = file.readlines()  
  
# Display each line.  
for line in lines:  
    print(line.strip())
```

Write a Python program to read the first three lines of `students.txt` using `readline()`.

```
# Open the file in read mode.  
with open("students.txt", "r") as file:  
  
    # Read the first line.  
    print(file.readline().strip())  
  
    # Read the second line.  
    print(file.readline().strip())  
  
    # Read the third line.  
    print(file.readline().strip())
```

`strip()` is used to **remove unwanted spaces and newline characters from the beginning and end of a string.**

For information only....

A **binary file** stores data in the form of **binary bytes** rather than normal human-readable text.

Examples of binary files include:

- Images such as `.jpg`, `.png`
- Videos such as `.mp4`
- Audio such as `.mp3`
- PDF files
- Executable files
- Some database and compressed files

A binary file does **not necessarily look readable** when viewed as raw data. It consists of bytes such as:

10101010 00101101 11001010 ...

However, in Python, binary data is commonly represented as bytes:

`b'\x89PNG\r\n\x1a\n...'`

Mode	Full Meaning	Purpose	If File Does Not Exist
rb	Read Binary	Reads data from a binary file	Error
wb	Write Binary	Writes binary data and overwrites existing data	Creates file
ab	Append Binary	Adds binary data at the end	Creates file
rb+	Read + Write Binary	Reads and writes binary data	Error
wb+	Write + Read Binary	Writes and reads binary data; overwrites existing data	Creates file
ab+	Append + Read Binary	Appends and reads binary data	Creates file